SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 10%

QUESTIONS: 05 **Total Marks:100**

Annexure A

Comparative Analysis

Code of Conduct:

I **Chalamala Sai Harshitha (192424346)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
C. Sai Harshitha

Annexure A

1. **Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.**

K-Nearest Neighbour (KNN) and **Case-Based Learning (CBL)** are learning approaches that solve a new problem by using previously stored examples or cases. In medical diagnosis, both can use information from previous patients to help diagnose a new patient. However, they differ in how they store previous information, measure similarity, and make predictions.

Medical Diagnosis Example

Consider a hospital database containing previous patient records with:

Age, Blood Pressure, Blood Sugar, Cholesterol, Symptoms → Diagnosis

When a new patient arrives, the patient's information is compared with previous patient records.

1. Memory Usage

KNN:

- Stores the training dataset and uses it directly during prediction.
- It does not create a separate predictive model during training.
- Therefore, memory requirements can become high when the medical dataset contains many patient records.

Case-Based Learning:

- Stores previous patient cases in a **case base**.
- Cases can be organized and less useful or redundant cases can be removed.
- Therefore, memory can be managed according to the usefulness of stored cases.

2. Similarity Measures

KNN:

- Usually measures similarity using mathematical distance.
- A commonly used measure is **Euclidean distance**:

$$d(x,y) = \sqrt{[(x1 - y1)^2 + (x2 - y2)^2 + ... + (xn - yn)^2]}$$

- A smaller distance indicates greater similarity.
- For example, a new patient whose blood pressure, blood sugar, and cholesterol values are close to another patient's values will have a smaller distance.

Case-Based Learning:

- Uses a similarity function to find the most relevant previous case.
- The similarity can be designed according to the problem domain.
- In medical diagnosis, important symptoms or medical attributes can be given greater importance rather than treating every attribute equally.

3. Prediction Strategy

KNN:

Suppose **K = 5** and the five nearest patients have:

- 3 patients → Disease A
- 2 patients → Disease B

KNN predicts **Disease A** because it uses **majority voting** among the nearest neighbours.

Case-Based Learning:

CBL generally follows the cycle:

Retrieve → Reuse → Revise → Retain

- **Retrieve:** Find a previous patient case similar to the new patient.
- **Reuse:** Use the diagnosis or treatment from the previous case.
- **Revise:** Modify the solution if the new patient's condition is different.
- **Retain:** Store the new case for possible future use.

4. Key Differences

Parameter	KNN	Case-Based Learning
Memory	Stores training examples	Stores previous cases in a case base
Similarity	Mainly distance-based	Can use domain-specific similarity
Prediction	Majority vote of nearest neighbours	Reuses and adapts a previous solution
Flexibility	Less flexible in adapting solutions	More flexible because cases can be revised
Medical use	Classifies patients based on similar patients	Uses previous medical cases to support diagnosis and treatment

Conclusion

KNN predicts using the nearest examples and majority voting, while CBL retrieves and adapts previous cases. Thus, KNN is suitable for direct classification, whereas CBL is more flexible for reusing previous medical cases.

2. **Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.**

Locally Weighted Regression (LWR) and **Radial Basis Function (RBF) Networks** are used for function approximation. Both give importance to nearby data points, but they differ in how locality is handled, computational cost, and smoothness of predictions.

Example Dataset

Consider a dataset showing the relationship between **advertising expenditure and sales**:

Advertising (X)	Sales (Y)
1	2
2	4
3	5
4	7
5	8
6	10

The objective is to predict sales for a new advertising value.

1. Comparison

Aspect	Locally Weighted Regression (LWR)	Radial Basis Function (RBF) Network
Locality	Gives higher weight to data points close to the query point.	Uses basis functions centred around selected data points.
Working	Fits a separate local regression model for each query.	Combines outputs of radial basis functions to make predictions.
Computational cost	Higher during prediction because a local model is calculated for each query.	Higher during training, but prediction is faster after training.
Smoothness	Depends on the weighting function and bandwidth.	Generally produces smooth predictions because basis functions overlap.
Model	Query-dependent and non-parametric.	Neural-network-based model.

2. Locality

LWR:

- Nearby observations receive higher weights.
- Distant observations receive lower weights.
- A new local model is created for each query point.

RBF Network:

- Uses centres and widths of radial basis functions.
- An input produces a stronger response when it is closer to a centre.

3. Computational Cost

LWR:

- Distance and weights are calculated for every new prediction.
- Can become expensive for large datasets.

RBF Network:

- Training requires finding suitable centres and learning weights.
- After training, prediction is generally faster.

4. Smoothness of Predictions

- **LWR:** Smoothness depends on the chosen bandwidth and weighting function.
- **RBF:** Usually gives smooth non-linear predictions because several basis functions contribute to the output.

Conclusion

LWR builds a local model for each query, while RBF Networks learn local basis functions during training. LWR is more computationally expensive during prediction, whereas RBF Networks generally provide faster and smooth predictions after training.

3. **Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.**

Naïve Bayes and Logistic Regression are supervised classification methods that can classify emails as Spam or Not Spam. Their main differences are in feature independence, probability estimation, computational efficiency, and performance with high-dimensional text data.

Example: Spam Detection

Suppose the dataset contains:

Email → Spam / Not Spam

Features can include words such as **"free", "offer", "win", "prize", "meeting", and "project".**

1. Comparison

Criteria	Naïve Bayes	Logistic Regression
Feature assumption	Assumes features are conditionally independent.	Does not assume feature independence.
Probability estimation	Estimates probabilities using Bayes' theorem.	Estimates probability using the sigmoid function.
Computational efficiency	Fast training and prediction.	Training is comparatively more computationally intensive.
High-dimensional data	Very suitable for text data with many features.	Also suitable for high-dimensional sparse text data.
Decision making	Selects the class with the highest probability.	Uses the predicted probability and a decision threshold.

2. Probability Estimation

Naïve Bayes:

$$P(\text{Class} \mid \text{Features}) = [P(\text{Features} \mid \text{Class}) \times P(\text{Class})] / P(\text{Features})$$

The class with the highest probability is selected.

Logistic Regression:

$$P(y = 1) = 1 / [1 + e^{(-z)}]$$

The output is a probability between **0 and 1**, which is used to classify the email.

3. Performance on High-Dimensional Data

Email datasets can contain **thousands of word features**.

- **Naïve Bayes:** Efficient and works well with many text features.
- **Logistic Regression:** Also performs well with high-dimensional and sparse features.

4. When Does One Perform Better?

Naïve Bayes performs better when:

- Fast training is required.
- The dataset is relatively small.
- Simple text classification is needed.

Logistic Regression performs better when:

- Feature relationships are important.
- A stronger discriminative decision boundary is required.
- Sufficient training data is available.

Conclusion

Naïve Bayes is faster and simpler for text classification, while Logistic Regression is more flexible because it does not assume feature independence.

4. **Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.**

Support Vector Machine (SVM) and **K-Nearest Neighbour (KNN)** are supervised learning algorithms used for classification. They differ mainly in their decision boundaries, scalability, sensitivity to noise, and computational complexity.

Example Dataset

Consider a dataset for classifying **flowers** using:

Sepal Length, Sepal Width, Petal Length, Petal Width → Flower Class

Suppose the task is to classify a new flower into **Class A, Class B, or Class C**.

1. Comparison

Aspect	SVM	KNN
Decision boundary	Learns an optimal boundary that separates classes.	Does not explicitly learn a boundary; classification depends on nearby points.
Scalability	Prediction is generally efficient after training, but training can be expensive for very large datasets.	Prediction becomes slower as the dataset grows because distances must be calculated to stored points.
Noise sensitivity	Generally, more robust, especially with a suitable margin and regularization.	More sensitive to noisy or irrelevant neighbouring points.
Training	Requires model training to find the separating hyperplane.	Requires very little training.
Prediction	Uses the learned decision function.	Uses the majority class among K nearest neighbours.

2. Decision Boundaries

SVM:

- Finds a **maximum-margin hyperplane** separating different classes.
- Only important training points called **support vectors** mainly determine the boundary.

KNN:

- Does not construct a fixed decision boundary during training.

- The boundary is formed implicitly based on the classes of nearby data points.

3. Computational Complexity

SVM:

- Training can be computationally expensive for large datasets.
- Prediction is relatively fast after the model has been trained.

KNN:

- Training is very simple because the data is mainly stored.
- Prediction can be expensive because distances to training points must be calculated.

4. Sensitivity to Noise

- **SVM:** The margin helps reduce the effect of individual noisy observations, particularly when appropriate regularization is used.
- **KNN:** A noisy or incorrectly labelled nearby point can directly affect the prediction.

5. Role of Kernel Functions in SVM

SVM can use kernel functions to handle non-linear classification problems.

Common kernels include:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF) Kernel

A kernel allows SVM to model complex non-linear relationships by representing the data in a higher-dimensional feature space without explicitly computing that transformation.

Conclusion

SVM learns a strong decision boundary and handles non-linear data using kernels, while KNN classifies data based on nearby examples. SVM is generally more suitable for complex classification boundaries, whereas KNN is simpler but can become expensive for large datasets.

5. **Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.**

K-Means and Hierarchical Clustering are unsupervised learning methods used to group data points without predefined class labels. They differ in cluster formation, computational cost, scalability, interpretability, and their ability to handle different cluster shapes.

Example Dataset

Consider a customer dataset containing:

Age, Annual Income, Spending Score

The actual groups of customers are unknown. The clustering algorithms are used to discover natural customer groups.

1. Comparison

Aspect	K-Means Clustering	Hierarchical Clustering
Cluster formation	Assigns data points to K clusters based on their nearest centroid.	Builds a hierarchy of clusters by repeatedly merging or splitting groups.
Number of clusters	K must be specified before clustering.	Number of clusters can be selected after viewing the dendrogram.
Computational cost	Generally faster and more efficient.	Generally more computationally expensive.
Scalability	Works well with large datasets.	Less suitable for very large datasets.
Interpretability	Easy to understand using cluster centroids.	Dendrogram provides a clear hierarchical view of relationships.
Cluster shape	Works best with compact, roughly spherical clusters.	Can identify more varied structures depending on the linkage method.
Result	Produces a fixed set of clusters.	Produces a hierarchy of nested clusters.

2. Cluster Formation

K-Means:

1. Select K initial centroids.
2. Assign each point to the nearest centroid.

3. Recalculate the centroids.
4. Repeat until the clusters become stable.

Hierarchical Clustering:

- Starts with individual data points as separate clusters.
- Similar clusters are progressively merged.
- The process produces a dendrogram showing the hierarchy.

3. Computational Cost and Scalability

- **K-Means:** Computationally efficient and suitable for large datasets because it repeatedly updates a limited number of centroids.
- **Hierarchical:** Requires comparisons between clusters during the hierarchy-building process, making it more expensive for large datasets.

4. Varying Cluster Shapes

- **K-Means:** Works best when clusters are approximately spherical and have similar characteristics.
- **Hierarchical:** Can handle more varied cluster structures depending on the selected linkage method and distance measure.

5. Impact of Choosing Number of Clusters

- **K-Means:** The value of K must be chosen before clustering, and an unsuitable K can produce poor groups.
- **Hierarchical:** Does not require the final number of clusters initially. A suitable number can be selected by cutting the dendrogram at an appropriate level.

Conclusion

K-Means is faster and more scalable, while Hierarchical Clustering provides a more interpretable hierarchy of groups. K-Means is suitable for large datasets with compact clusters, whereas Hierarchical Clustering is useful when the cluster structure is unknown.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**15**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**20**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand